Assignment 6 | Visitor Pattern

Berdibek Meiirbek | SE-2528 | Advanced File System

1. Introduction

The program models a file system in memory. The same tree supports three separate operations: total byte size, filename-and-size search, and file/directory statistics. Visitor keeps these operations outside the element classes, so another operation can be added without changing the existing elements [1, 2].

Element defines the common contract. FileNode stores a name and non-negative long size; DirectoryNode owns children. The abstract Node base shares name validation and internal parent bookkeeping. Main builds the sample tree and runs all three visitors.

2. UML class diagram

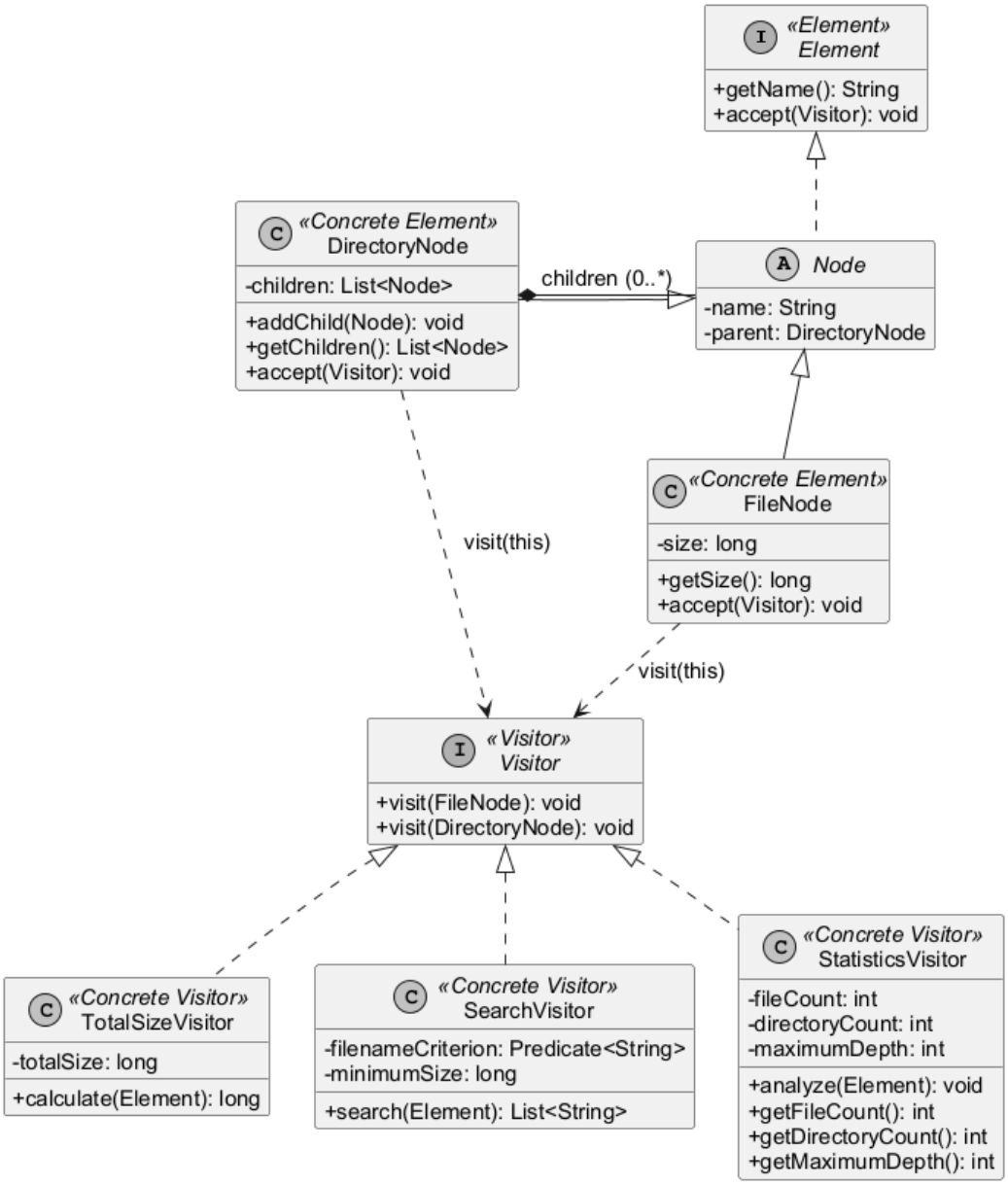


Figure 1. PlantUML diagram of the final code. Hollow triangles show inheritance or interface realization; the filled diamond shows directory ownership. Both concrete elements dispatch to Visitor. Only relevant members are shown [3].

3. Clean Code examples

3.1 Meaningful names

filesystem/SearchVisitor.java (lines 10-11)

```
private final Predicate<String> filenameCriterion;
private final long minimumSize;
```

filenameCriterion and minimumSize describe both search constraints directly. Search combines the configured predicate with an inclusive size comparison: size >= minimumSize.

3.2 Single Responsibility

filesystem/TotalSizeVisitor.java (lines 16-18)

```
public void visit(FileNode file) {
    totalSize = Math.addExact(totalSize, file.getSize());
}
```

This visitor only accumulates file sizes; directories contribute no bytes. Search and statistics have their own visitor classes, while the elements maintain the tree and its integrity. Math.addExact rejects long overflow.

3.3 Small focused methods

filesystem/FileNode.java (lines 16-18)

```
public long getSize() {
    return size;
}
```

getSize() returns one value without performing traversal or calculations. The statistics getters follow the same approach, keeping each method easy to read and explain.

3.4 Encapsulation

filesystem/DirectoryNode.java (lines 33-35)

```
public List<Node> getChildren() {
    return List.copyOf(children);
}
```

The returned snapshot cannot change the internal collection. addChild() rejects duplicate sibling names, existing parent ownership, self-links and ancestor cycles before mutation. Negative file sizes and invalid path-segment names are rejected at construction.

3.5 No type-checking chains

filesystem/FileNode.java (lines 21-23)

```
public void accept(Visitor visitor) {
    Objects.requireNonNull(visitor, "visitor").visit(this);
}
```

Double dispatch first selects the concrete element's accept(). Inside it, the concrete type of this selects the visit overload, and the actual visitor supplies that operation. DirectoryNode uses the same delegation. No instanceof, getClass(), reflection or casts select the element behavior.

4. Demonstration and verification

The workspace tree contains readme.txt (100 bytes), docs/report.txt (240), docs/notes.txt (80), docs/drafts/plan.txt (120), images/logo.png (500), and an empty directory. Main searches for .txt files of at least 100 bytes. Actual output from the JDK 17 run:

```
Total size: 1040 bytes
Matches (.txt, >= 100 bytes):
  docs/drafts/plan.txt
  docs/report.txt
  readme.txt
Statistics: files=5, directories=5, maximum depth=3
```

JUnit 5 verification completed with **BUILD SUCCESS: 24 tests, 0 failures, 0 errors, 0 skipped.** The test fixtures are separate from the demonstration data.

Test class (package filesystem)	Tests	Verification
VisitorTest	12	T1-T4, T6-T7: dispatch, nested sizes, both filters, sorted relative paths, empty/no-match cases and reuse.
TreeIntegrityTest	8	T8: invalid sizes/names, duplicates, ownership, cycles, child snapshots and valid nesting.
StatisticsVisitorTest	4	T5-T7: counts, root-relative depth, empty directories and repeated runs.

Run from the folder containing pom.xml:

```
mvn test
mvn -q compile exec:java
```

5. Design choices and conclusion

Visitors own recursion: each directory visit calls child.accept(this) once per child. accept() itself does not traverse. Start sequential runs through calculate(root), search(root) or analyze(root); these reset all accumulated state. Direct accept/visit calls are callbacks. Do not mutate the tree during traversal.

Search restores the directory prefix after each subtree. Paths omit the chosen directory root, use / and sort lexicographically; a file root uses its filename. Depth starts at zero, increases for each child level, and includes empty directories. A directory root is included in the directory count.

Commit fc6b24b adds StatisticsVisitor, tests and its client call after commit 5fe0340 without changing Element, Visitor, Node, FileNode, DirectoryNode or either accept(). New operations therefore fit the existing contract. A new element type requires another visit overload and updates to every visitor. The trade-off of visitor-owned recursion is a short repeated traversal loop and stack usage proportional to depth.

6. GitHub repository

https://github.com/Mikosh001/Assignment_6_Visitor_Pattern

References

[1] Astana IT University. Assignment 6 - Visitor Pattern instructions.
[2] Refactoring.Guru. Visitor.
[3] PlantUML. Class diagram.
[4] OpenAI Codex. Assistance with implementation, report drafting and verification.